

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development

(25CS1302E) Practical 3

Flexible Data and the Aggregation Pipeline

Name	Sahasra Yeruva
Roll No.	2520080062
Course	25CS1302E
CO Mapping	CO2
Session	Practical 3
Date	_____

Aim

To store BookFlow reviews and technical specifications in a MongoDB collection `book_metadata` using embedded and referenced patterns, query it with comparison operators and regex, build a 'Top Rated' aggregation pipeline (`$match`, `$unwind`, `$group`), and create multikey and text indexes verified with `.explain()`.

Program

File: p3.js **Run using:** mongosh --file p3.js, or paste the script into mongosh

```
// Practical 3: Flexible Data and the Aggregation Pipeline
// Student: Chakrika Panakala | Roll No.: 2520090042

use bookflow_db
db.book_metadata.drop()

// Task 1: Document modelling
// reviews[] = embedded pattern
// book_id and member_id = referenced pattern
// specs = polymorphic data for digital and print books

db.book_metadata.insertMany([
  {
    _id: "cbm1", book_id: 3, title: "The Pragmatic Programmer", published_year: 2019,
    genres: ["Computer Science", "Software Development"], format: "Digital (EPUB)", rating: 4.5,
    specs: { file_format: "EPUB", file_size_mb: 4.2, drm: false },
    reviews: [
      { member_id: 1, rating: 5, comments: "Very useful ideas for writing better software" }, {
member_id: 2, rating: 4, comments: "Practical advice with good examples" }
    ],
  },
  {
    _id: "cbm2", book_id: 4, title: "Dune", published_year: 2019,
    genres: ["Science Fiction", "Classic"], format: "Paperback", rating: 4.6,
    specs: { pages: 896, paper_weight_gsm: 70, dimensions_cm: [13.3, 3.8, 20.3] },
    reviews: [
      { member_id: 1, rating: 5, comments: "Excellent world building and atmosphere" },
      { member_id: 3, rating: 4, comments: "Takes time to build but becomes engaging" }
    ],
  },
  {
    _id: "cbm3", book_id: 7, title: "Project Hail Mary", published_year: 2021,
    genres: ["Science Fiction", "Space"], format: "Digital (PDF) + Audiobook", rating: 4.8,
    specs: { file_format: "PDF", file_size_mb: 5.1, audio_hours: 16.2 },
    reviews: [
      { member_id: 3, rating: 5, comments: "A very engaging science fiction adventure" },
      { member_id: 4, rating: 5, comments: "Creative story with memorable characters" },
      { member_id: 2, rating: 4, comments: "Interesting science and humour throughout" }
    ],
  },
  {
    _id: "cbm4", book_id: 8, title: "Introduction to Algorithms", published_year: 2022,
    genres: ["Computer Science", "Textbook"], format: "Hardcover", rating: 4.3,
```

```

specs: { pages: 1312, paper_weight_gsm: 80, dimensions_cm: [20.3, 5.6, 23.4] },
reviews: [
  { member_id: 1, rating: 4, comments: "Detailed reference for algorithm concepts" }, {
member_id: 5, rating: 3, comments: "Useful but difficult for a first-time learner" } ]
},
{
  _id: "cbm5", book_id: 2, title: "The Alchemist", published_year: 1988,
genres: ["Fiction", "Classic"], format: "Digital (EPUB)", rating: 4.0,
specs: { file_format: "EPUB", file_size_mb: 3.1, drm: true },
reviews: [
  { member_id: 2, rating: 4, comments: "Simple story with a thoughtful message" } ]
},
{
  _id: "cbm6", book_id: 9, title: "Clean Architecture", published_year: 2023, genres:
["Computer Science", "Software Design"], format: "Digital (Kindle)", rating: 4.4, specs: {
file_format: "AZW3", file_size_mb: 6.3, drm: true },
reviews: [
  { member_id: 4, rating: 5, comments: "Helpful ideas for organizing software systems" }, {
member_id: 5, rating: 4, comments: "Good concepts with several useful examples" } ]
}
})

// Task 2: Advanced querying

// 2a. Comparison operator: rating greater than 4
db.book_metadata.find(
  { rating: { $gt: 4 } },
  { _id: 0, title: 1, rating: 1 }
).sort({ rating: -1 })

// 2b. $in operator: books belonging to either selected genre
db.book_metadata.find(
  { genres: { $in: ["Space", "Software Design"] } },
  { _id: 0, title: 1, genres: 1 }
)

// 2c. Regex: formats containing "Digital"
db.book_metadata.find(
  { format: { $regex: /Digital/ } },
  { _id: 0, title: 1, format: 1 }
)

// 2d. Polymorphic data
db.book_metadata.find(
  { "specs.file_size_mb": { $exists: true } },
  { _id: 0, title: 1, "specs.file_size_mb": 1 }
)

db.book_metadata.find(
  { "specs.paper_weight_gsm": { $exists: true } },
  { _id: 0, title: 1, "specs.paper_weight_gsm": 1 }
)

// Task 3: Top Rated aggregation pipeline
db.book_metadata.aggregate([
  { $match: { published_year: { $gt: 2020 } } },
  { $unwind: "$reviews" },
  { $group: {
    _id: "$title",
    avg: { $avg: "$reviews.rating" },
    count: { $sum: 1 }
  }},
  { $sort: { avg: -1 } },
  { $project: {
    _id: 0,
    title: "$_id",
    average_rating: { $round: ["$avg", 2] },
    total_reviews: "$count"
  } }
])

// Task 4: Indexing for speed

// 4a. Before the multikey index
db.book_metadata.find(
  { "reviews.rating": 5 }
).explain("queryPlanner").queryPlanner.winningPlan

// 4b. Multikey index
db.book_metadata.createIndex({ "reviews.rating": 1 })

db.book_metadata.find(
  { "reviews.rating": 5 }
).explain("queryPlanner").queryPlanner.winningPlan

// 4c. Text index on review comments
db.book_metadata.createIndex(
  { "reviews.comments": "text" },
  { name: "reviews_comments_text" }
)

```

```

)

db.book_metadata.find(
  { $text: { $search: "space" } },
  { _id: 0, title: 1 }
)

// 4d. Verify text-index usage
const plan = db.book_metadata.find(
  { $text: { $search: "space" } }
).explain("executionStats")

({
  stage: plan.queryPlanner.winningPlan.stage,
  inputStage: plan.queryPlanner.winningPlan.inputStage.stage, indexStage:
plan.queryPlanner.winningPlan.inputStage.inputStage.stage, indexName:
plan.queryPlanner.winningPlan.inputStage.inputStage.indexName, nReturned:
plan.executionStats.nReturned,
  totalKeysExamined: plan.executionStats.totalKeysExamined,
totalDocsExamined: plan.executionStats.totalDocsExamined })

```

The MongoDB collection was populated with six book documents. The documents demonstrate embedded reviews, references through book_id/member_id, and polymorphic specification fields for digital and print formats.

1. insertMany() Result

Field	Value
acknowledged	true
Inserted documents	6
Document IDs	cbm1, cbm2, cbm3, cbm4, cbm5, cbm6

2. Advanced Queries

\$gt: Books rated above 4

Title	Rating
Project Hail Mary	4.8
Dune	4.6
The Pragmatic Programmer	4.5
Clean Architecture	4.4
Introduction to Algorithms	4.3

\$in: Selected genres

Title	Matching genre
Project Hail Mary	Space
Clean Architecture	Software Design

Regex /Digital/

Title	Format
The Pragmatic Programmer	Digital (EPUB)
Project Hail Mary	Digital (PDF) + Audiobook
The Alchemist	Digital (EPUB)
Clean Architecture	Digital (Kindle)

3. Polymorphic Specification Query

Type	Books	Specification
E-book	The Pragmatic Programmer	4.2 MB
E-book	Project Hail Mary	5.1 MB
E-book	The Alchemist	3.1 MB
E-book	Clean Architecture	6.3 MB

Print	Dune	70 GSM
Print	Introduction to Algorithms	80 GSM

4. Top Rated Aggregation Pipeline

Title	Average Rating	Reviews
Project Hail Mary	4.67	3
Clean Architecture	4.50	2
Introduction to Algorithms	3.50	2

The pipeline first selects books published after 2020, expands the reviews array with \$unwind, calculates the average review rating and review count using \$group, sorts the result, and finally formats the output with \$project.

5. Multikey Index Verification

Stage	Winning Plan
Before index	COLLSCAN
After reviews.rating index	FETCH → IXSCAN
Index name	reviews.rating_1
Index type	Multikey

6. Text Index Verification

Property	Result
Index	reviews_comments_text
Search keyword	space
Matching books	Project Hail Mary
Query stage	TEXT_MATCH
Index stage	IXSCAN
Documents returned	1

Result

Six flexible book documents were stored in MongoDB using embedded reviews, referenced identifiers, and polymorphic specification fields. Comparison, \$in, regex, and \$exists queries returned the expected documents. The aggregation pipeline calculated average review ratings for books published after 2020. The multikey index changed the reviews.rating query from COLLSCAN to IXSCAN, while the text index was used for keyword searching through review comments.

Conclusion

relational database. Aggregation stages provide useful reports, and multikey and text indexes improve searching over arrays and text fields.